

使用 uv 高效管理 Python 环境

Print to PDF

Contents

- 5.1. 引言：为什么需要环境管理？
- 5.2. uv：新一代的 Python 环境与包管理工具
- 5.3. uv 的安装与配置
- 5.4. 针对本讲义项目的环境搭建流程
- 5.5. 日常使用与维护

5.1. 引言：为什么需要环境管理？

在进行 Python 项目开发时，我们通常会依赖许多第三方库（例如本讲义中用到的 `numpy`, `pandas`, `coolprop` 等）。不同项目可能需要不同版本的库，甚至需要不同版本的 Python 解释器。如果在系统全局环境中安装所有库，很快就会导致版本冲突和混乱，使得项目难以维护和复现。

为了解决这个问题，我们引入了 **虚拟环境** 的概念。

5.1.1. 虚拟环境 (`venv`) 的基本概念

虚拟环境是 Python 用于创建项目隔离开发环境的标准机制。您可以把它想象成一个独立的、轻量级的 Python 安装副本，它位于您的项目文件夹内。

- **隔离性**: 在虚拟环境中安装的库仅对该环境有效，不会影响系统全局或其他项目。
- **可复现性**: 每个项目都有其独立的环境和依赖列表（通常是 `requirements.txt` 或 `pyproject.toml` 文件），任何人拿到您的项目后，都可以根据这个列表创建一模一样的环境，确保代码能够顺利运行。

在现代 Python 中，我们可以使用内置的 `venv` 模块来创建虚拟环境：

```
# 1. 创建一个名为 .venv 的虚拟环境
python -m venv .venv

# 2. 激活环境 (Windows PowerShell)
.venv\Scripts\Activate.ps1
```

激活后，您会发现终端提示符前面出现了 `(.venv)` 字样，表示您现在正处于这个虚拟环境中。此时，使用 `pip` 安装的任何包都将被安装到 `.venv` 文件夹内。

5.2. `uv`: 新一代的 Python 环境与包管理工具

虽然 `venv` 和 `pip` 是标准工具，但它们的性能在处理大型项目和复杂依赖时可能会变慢。`uv` 是由 `ruff` 的作者开发的一款用 Rust 编写的、速度极快的 Python 打包工具，旨在成为 `pip` 和 `venv` 的替代品。

`uv` 的核心优势:

- 极速:** `uv` 的依赖解析和安装速度比 `pip` 快 10-100 倍。
- 一体化:** 它将虚拟环境创建、包安装、依赖锁定等功能集于一身。
- 兼容性:** `uv` 完全兼容 `pyproject.toml` 和 `requirements.txt` 文件，可以无缝集成到现有项目中。

对于本讲义项目，我们将使用 `uv` 来统一和加速环境管理流程。

5.3. `uv` 的安装与配置

`uv` 的安装非常简单。在 Windows 系统上，您只需在 PowerShell 中运行以下命令：

```
irm https://astral.sh/uv/install.ps1 | iex
```

安装程序会自动将 `uv` 的路径添加到您的系统环境变量中。安装完成后，重新打开一个终端，运行 `uv --version` 来验证安装是否成功。

5.4. 针对本讲义项目的环境搭建流程

现在，我们将一步步使用 `uv` 为本讲义项目创建一个干净、独立且完整的开发环境。

5.4.1. 第一步：创建虚拟环境

首先，请确保您已经通过 `git` 克隆了本讲义的仓库，并进入了项目根目录。

本项目的 `.python-version` 文件指定了所需的 Python 版本（3.12）。`uv` 可以自动识别此文件。运行以下命令来创建虚拟环境：

```
# uv 会自动查找 .python-version 文件，并使用对应的 Python 版本
# 如果找不到，它会使用系统默认的 python3
# 虚拟环境将被创建在项目根目录下的 .venv 文件夹中
uv venv
```

5.4.2. 第二步：激活虚拟环境

与 `venv` 一样，创建好的环境需要被激活才能使用。

```
# 在 Windows PowerShell 中激活
.venv\Scripts\Activate.ps1
```

激活成功后，您的终端提示符将显示 `(.venv)`。

5.4.3. 第三步：安装项目依赖

本讲义的所有依赖项都定义在 `pyproject.toml` 文件中。`uv` 可以直接读取这个文件并安装所有必需的库。我们使用 `-e .` 的方式进行安装，这被称为“可编辑模式”，对于开发项目非常方便。

```
# 确保你已经激活了 .venv 环境
# uv 会读取 pyproject.toml 文件并安装所有依赖
uv pip install -e .
```

`uv` 会以惊人的速度下载和安装 `cadquery`, `coolprop`, `jupyter-book` 等所有库。等待命令执行完毕，您的开发环境就完全准备好了！

5.5. 日常使用与维护

5.5.1. 运行 Jupyter Book

环境搭建完成后，您可以直接使用 `jupyter-book` 命令来构建本书。

```
# 构建本书的 HTML 文件
# 这会在 modules/_build/html/ 目录下生成网站
jupyter-book build modules -all
```

5.5.2. 预览生成的内容

构建完成后，您可以使用 Python 内置的 HTTP 服务器来本地预览您的书籍。

```
# 首先，切换到包含 HTML 文件的目录
cd modules/_build/html

# 然后，启动一个简单的 HTTP 服务器
# 默认情况下，它会在 8000 端口提供服务
python -m http.server --bind 0.0.0.0 80
```

或者，可以给 `http.server` 设定启动目录：

```
python -m http.server --directory modules/_build/html --bind 0.0.0.0 8000
```

这样就不需要切换目录了。

之后，您可以在浏览器中打开 `http://localhost:8000` 来查看您的书籍。

或者，可以直接使用浏览器打开 `modules/_build/html/index.html` 文件进行预览。

5.5.3. 添加新的依赖库

如果在后续学习中需要添加新的库（例如 `seaborn`），推荐的做法是：

1. 使用 `uv` 安装该库：

```
uv add seaborn
```

2. 手动将库名（`"seaborn"`）添加到 `pyproject.toml` 文件的 `dependencies` 列表中，以便环境可以被他人复现。

5.5.4. 同步环境

如果 `pyproject.toml` 文件发生了变化（例如，您通过 `git pull` 更新了项目，其中包含了新的依赖），您可以使用 `sync` 命令来确保您的虚拟环境与配置文件保持一致。

```
# sync 命令会安装缺失的包，并移除不再需要的包
uv sync pyproject.toml
```

通过以上步骤，您就可以利用 `uv` 高效、可靠地管理本讲义的 Python 环境了。